

Terraform Real-World Example

Three-Tier Architecture (Dev, QA, Staging)

Three-Tier Architecture (Dev, QA, Staging)

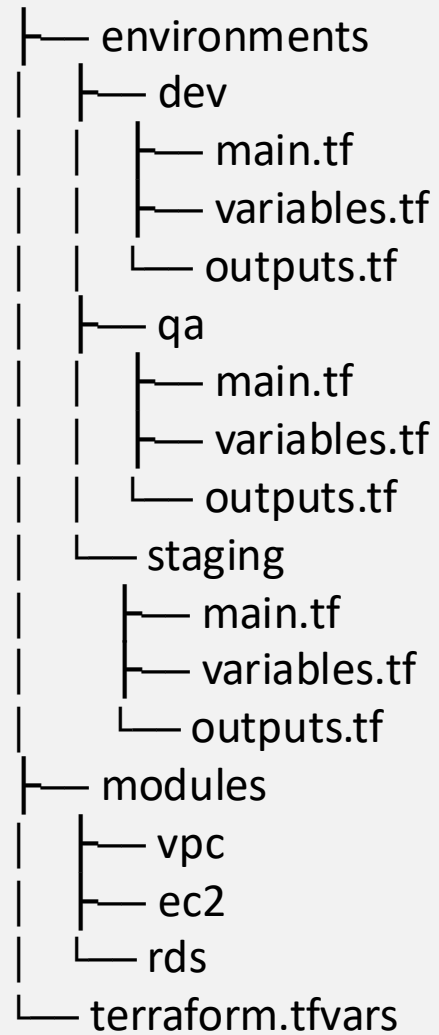
Let's create a complete **Terraform project** to deploy a three-tier architecture with environments for dev, qa, and staging. This example covers most Terraform concepts, including variables, modules, outputs, remote state, and provisioners.



Key Highlights of the Project:

- ✓ Multi-environment setup with isolated configurations for **Dev, QA, and Staging**.
- ✓ Modular approach for easy reuse and scalability.
- ✓ Secure state management with **S3, DynamoDB locking**, and **encryption using KMS**.
- ✓ Infrastructure provisioning using Terraform with automated **EC2, VPC, and RDS** setups.

Project Structure



Modules Overview:

- **VPC Module:** Creates a secure VPC with public subnets.
- **EC2 Module:** Deploys scalable web servers using EC2 instances.
- **RDS Module:** Sets up a secure MySQL database instance.

Ensuring Security for Terraform State:

- Remote state stored securely in **AWS S3** with versioning and encryption.
- State locking using **DynamoDB** to prevent concurrent modifications.
- Encrypted state files with **AWS KMS**.
- Restricted access via **IAM policies**.



Sample Configurations:

Module: VPC (modules/vpc/main.tf)

```
resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr
  tags      = { Name = var.vpc_name }
}

resource "aws_subnet" "public" {
  count      = 2
  vpc_id     = aws_vpc.main.id
  cidr_block = var.public_subnets[count.index]
  map_public_ip_on_launch = true
}
```

Module: EC2 (modules/ec2/main.tf)

```
resource "aws_instance" "web" {  
  count      = var.instance_count  
  ami       = var.ami_id  
  instance_type = var.instance_type  
  
  tags = {  
    Name = "${var.env}-web-server-${count.index}"  
  }  
}
```

Module: RDS (modules/rds/main.tf)

```
resource "aws_db_instance" "db" {  
  allocated_storage = 20  
  engine            = "mysql"  
  instance_class    = "db.t3.micro"  
  username          = var.db_user  
  password          = var.db_password  
  publicly_accessible = false  
}
```

Environment Config (environments/dev/main.tf)

```
provider "aws" {  
  region = "us-east-1"  
}  
  
module "vpc" {  
  source      = "../modules/vpc"  
  vpc_cidr    = "10.0.0.0/16"  
  public_subnets = ["10.0.1.0/24", "10.0.2.0/24"]  
  vpc_name    = "dev-vpc"  
}  
  
module "ec2" {  
  source      = "../modules/ec2"  
  ami_id      = "ami-12345678"  
  instance_type = "t3.micro"  
  instance_count = 2  
  env         = "dev"  
}
```


Variables (environments/dev/variables.tf)

```
variable "instance_count" { default = 2 }  
variable "ami_id" { default = "ami-12345678" }  
variable "vpc_cidr" { default = "10.0.0.0/16" }  
variable "public_subnets" { type = list(string) }  
variable "env" { default = "dev" }
```

Outputs (environments/dev/outputs.tf)

```
output "vpc_id" {  
  value = module.vpc.vpc_id  
}  
  
output "ec2_instance_ids" {  
  value = module.ec2.web_instance_ids  
}
```

Remote State Configuration

```
terraform {  
  backend "s3" {  
    bucket      = "my-terraform-state"  
    key         = "dev/terraform.tfstate"  
    region      = "us-east-1"  
    dynamodb_table = "terraform-lock"  
    encrypt     = true  
  }  
}
```

Provisioners Example

```
resource "aws_instance" "web" {  
  provisioner "local-exec" {  
    command = "echo 'Server ready' >> /tmp/server.log"  
  }  
}
```

How to Deploy:

1. Initialize the Terraform configuration:

```
terraform init
```

2. Preview the infrastructure changes

```
terraform plan
```

3. Apply the changes to create resources:

```
terraform apply
```

4. Destroy the infrastructure if needed:

```
terraform destroy
```